

0 vimrc	2	convex-hull.cpp	9
vimrc	2	cross-product.cpp	9
1 Number Theory	2	extend-li-chao-tree.cpp	10
Exgcd.cpp	2	li-chao-tree.cpp	10
catalan.cpp	2	max-man-dis.cpp	10
findPrime.cpp	2	minimum-euc-dis.cpp	10
matrixMultiple.cpp	2	pick-theorem.cpp	11
ntt.cpp	2	ray-casting.cpp	11
system-of-linear-eqn.cpp	3	動態凸包.cpp	11
2 Tree	3	多邊形重心計算.cpp	11
LCA.cpp	3	找三點組成最大圓.cpp	12
MST.cpp	3	極角排序.cpp	12
binary-lifing.cpp	4	5 Data Structure	12
find-centriod.cpp	4	Array Version of Linked List.cpp	12
hld.cpp	4	Binary Index Tree.cpp	13
small-to-large.cpp	4	Disjoint Set Union.cpp	13
tree-diameter.cpp	4	Segment Tree.cpp	13
3 Graph	5	persistence-seg-tree.cpp	13
2-sat.cpp	5	splay-tree.cpp	13
BellmanFord.cpp	5	6 Not that important	14
Dijkstra.cpp	5	EPS.cpp	14
Edmond.cpp	5	Gauce Elimination.cpp	14
articulation-point.cpp	5	Joseph.cpp	14
bridge.cpp	6	7 New	15
dinic.cpp	6	Convex_hull_trick.cpp	15
dominator-tree.cpp	6	Ray_casting.cpp	15
floyd-warshall.cpp	7	block_cut_tree.cpp	15
hierholzer.cpp	7	cross_dot_product.cpp	16
hopcroft-karp.cpp	7	dec_string_to_bitset.cpp	16
kuhn.cpp	7	dominator_tree.cpp	16
mcmf.cpp	7	fft.cpp	16
scc.cpp	8	fwt.cpp	17
toposort.cpp	8	li_chao_tree.cpp	17
4 Geometry	8	shoelace.cpp	18
Determine whether a polygon is convex.cpp	8	suffix_automaton.cpp	18
Rotating Calipers 求凸包直徑.cpp	8	suffixarray_and_lcp.cpp	18
all solutions for Diophantine Equation in given		xor_basis.cpp	19
range.cpp	9	zfunction_and_kmp.cpp	19

0 vimrc

vimrc

```

1 "variable
2 let mapleader = ','
3
4 "
5 set number
6 set relativenumber
7 set noexpandtab
8 set shiftwidth=4
9 set tabstop=4
10
11 inoremap kj <Esc>
12 inoremap {<CR> {<Left><CR><Esc>O
13 tnoremap kj <C-]><C-n>
14 nnoremap <leader>c :w<CR>:!g++ -std=c++20 % -o %:r &&
↵ ./%:r<CR>
15 nnoremap [ [
16 nnoremap ] ]

```

1 Number Theory

Exgcd.cpp

```

1 array<int,2> exgcd(int a, int b){
2     if(b == 0) return array<int,2>{1,0};
3     auto ans = exgcd(b, a % b);
4     return array<int,2>{ans[1], ans[0] - a / b *
↵ ans[1]};
5 }
6

```

catalan.cpp

```

1 C_0=1
2 C_n = $\sum C_i * C_{n-1-i}$ = $\frac{1}{n+1} C_n^{2n}$

```

findPrime.cpp

```

1 constexpr ll mxN=1e6+1;
2 mobius[1]=1;
3 forn(i,2,mxN){
4     if(!prime[i])
↵ prime[i]=i,mobius[i]=-1,etf[i]=i-1,st[++sz]=i;
5     forn(j,1,sz+1){
6         if(i*st[j]>mxN) break;
7         prime[i*st[j]]=st[j];
8         if(i%st[j]==0){
9             mobius[i*st[j]]=0;
10            etf[i*st[j]]=etf[i]*st[j];
11            break;
12        }
13        else mobius[i*st[j]]=mobius[i]*-1,etf[i*st[j]]
↵ [j]=etf[i]*(st[j]-1);
14    }
15 }

```

matrixMultiple.cpp

```

1 vector<vector<int>> matrixMultiply(const
↵ vector<vector<int>>& A, const vector<vector<int>>& B){
2     int rowA = A.size();
3     int colA = A[0].size();
4     int colB = B[0].size();

```

```

vector<vector<int>> C(rowA, vector<int>(colB, 0));
for (int i = 0; i < rowA; ++i) {
    for (int j = 0; j < colB; ++j) {
        for (int k = 0; k < colA; ++k) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
return C;
}

```

ntt.cpp

```

using ll=long long;
class FFT{
public:
    ll mod,root,root_inv,maX,M,M_inv;
    FFT(){
        FFT(ll a,ll b,ll c,ll d,ll e,ll
↵ f):mod(a),root(b),root_inv(c),maX(d),M_
↵ (e),M_inv(f){
        ll fsp(ll a,ll m){
            ll ans=1;
            while(m){
                if(m&1) ans=(ans*a)%mod;
                a=(a*a)%mod,m>>=1;
            }
            return ans;
        }
        void fft(ll sz,vc<ll> &arr,ll inv){
            for(ll i=1,j=0;i<sz;++i){
                ll bit=sz>>1;
                for(;j&bit;bit>>=1)
↵ j^=bit; j^=bit;
                if(i<j)
↵ swap(arr[i],arr[j]);
            }
            for(ll l=2;l<=sz;l<=(l<=1)){
                ll wlen=inv?root_inv:root;
                for(ll i=l;i<maX;i<=(l<=1)
↵ wlen=(wlen*wlen)%mod;
                for(ll i=0;i<sz;i+=l){
                    ll w=1;
                    for(ll
↵ j=0;j<l/2;++j){
                        ll x=arr[i]
↵ +j],y=
↵ (w*arr[
↵ [i+j+l]
↵ /2])%m
↵ od;
                        arr[i+j]=
↵ x+y<mo
↵ d?x+y:
↵ x+y-mo
↵ d);
                        arr[i+j+l/
↵ 2]=(x-
↵ y>=0?x
↵ -y:x-y
↵ +mod);
                        w=(w*wlen)
↵ %mod;
                    }
                }
            }
            if(inv){
                ll n_1=fsp(sz,mod-2);
                forn(i,0,sz){
                    arr[i]=(arr[i]*n_1
↵ )%mod;
                }
            }
        }
};
constexpr ll MM=1002772198720536577;

```

```

42 ll mul(ll a,ll b){
43     __int128 aa=a,bb=b,cc;
44     cc=(aa*bb)%MM;
45     return (ll)cc;
46 }
47 void solve(){
48     FFT fft1(998244353,15311432,469870224,1<<23,1004531);
49     FFT fft2(1004535809,702606812,700146880,1<<21,99833);
50     ll n,m;
51     cin >> n >> m;
52     vc<ll> a(n+1),b(m+1);
53     for(auto &v:a) cin >> v;
54     for(auto &v:b) cin >> v;
55     ll sz=1;
56     for(;sz<a.size()+b.size();sz<=<=1);
57     a.resize(sz),b.resize(sz);
58     vc<ll> A=A,B=b;
59     fft1.fft(sz,a,0);
60     fft1.fft(sz,b,0);
61     forn(i,0,sz) a[i]=(a[i]*b[i])%fft1.mod;
62     fft1.fft(sz,a,1);
63
64     fft2.fft(sz,A,0);
65     fft2.fft(sz,B,0);
66     forn(i,0,sz) A[i]=(A[i]*B[i])%fft2.mod;
67     fft2.fft(sz,A,1);
68
69     forn(i,0,n+m+1) cout <<
        ↪ (mul(a[i],fft1.M*fft1.M_inv) +
        ↪ mul(A[i],fft2.M*fft2.M_inv))%MM << "
        ↪ \n"[i==n+m];
70 }
71 int main()
72 {
73     fast_io;
74     int testcase;
75     // cin>>testcase;
76     testcase=1;
77     while(testcase--){
78         solve();
79         return 0;
80 }

```

system-of-linear-eqn.cpp

```

1 constexpr ll mxN=505;
2 ll n,m;
3 ll inv(ll a,ll m){ll ans; for(ans=1;m;ans=(m&1?(ans*a)%mdl
    ↪ 1:ans),a=(a*a)%mdl1,m>=>=1); return
    ↪ ans;}
4 void solve(istream &cin){
5     cin>>n>>m;
6     vc<ll> res(m,0);
7     vc<vc<ll>> a(n,vc<ll>(m+1));
8     forn(i,0,n) forn(j,0,m+1) cin>>a[i][j];
9     ll l=0;
10    for(ll i=0;i<m && l<n;++i){
11        if(!a[l][i]) forn(j,l+1,n)
            ↪ if(a[j][i]){swap(a[j],a[l]); break;}
12        if(!a[l][i]) continue;
13        ll c=inv(a[l][i],mdl1-2);
14        for(auto&v:a[l]) v=(v*c)%mdl1;
15        forn(j,l+1,n){
16            ll co=a[j][i];
17            forn(k,i,m+1) a[j][k]=(a[j][k]-co&
            ↪ *a[l][k])%mdl1+mdl1)%mdl1;
18        }
19        l++;
20    }
21    forn(i,n-1,0){
22        ll j=0;
23        while(j<m+1 && !a[i][j]) j++;
24        if(j==a[i].size()) continue;
25        if(j+1==a[i].size()){cout << -1 << '\n';
            ↪ return;}

```

```

26     ll u=a[i][m];
27     forn(k,j+1,m)
        ↪ u=((u-a[i][k]*res[k])%mdl1+mdl1)%mdl1;
28     res[j]=u;
29 }
30     forn(i,0,m) cout << res[i] << " \n"[i==m-1];
31 }
32 int main()
33 {
34     fast_io;
35     ll testcase;
36     // cin>>testcase;
37     testcase=1;
38     while(testcase--){
39         solve(cin);
40         return 0;
41 }

```

2 Tree

LCA.cpp

```

1
2 ll pa[mxN][20],d[mxN]{};
3 vc<ll> adj[mxN]
4 void dfs(ll u=1,ll p=0){d[u]=d[p]+1,pa[u][0]=p;
    ↪ for(auto&v:adj[u]) if(v!=p) dfs(v,u);}
5 ll lca(ll u,ll v){
6     if(d[u]<d[v]) swap(u,v); forr(i,20,0)
    ↪ if(d[u]-(1<<i)>=d[v]) u=pa[u][i]; if(u==v) return
    ↪ u;
7     forr(i,20,0) if(pa[u][i]!=pa[v][i])
    ↪ u=pa[u][i],v=pa[v][i]; return pa[u][0];
8 }
9
10 void init(int n) {
11     dfs(1, 0, 1);
12     for (int j = 1; j < 20; j++) {
13         for (int i = 1; i <= n; i++) {
14             parent[i][j] = parent[parent[i][j-1]][j-1];
15         }
16     }
17 }

```

MST.cpp

```

1 //kruskal
2 //vertex is 1 index
3 #define mls multiset
4
5 ll n,pa[mxN]; //memset(pa,-1,sizeof(pa));
6 mls<ar<ll,3>> adj;
7
8 ll find(ll x){return pa[x]<0?x:pa[x]=find(pa[x]);}
9 void un(ll x,ll y){
10     x=find(x),y=find(y); if(x==y) return;
11     if(-pa[x]>-pa[y]) swap(x,y); pa[y]+=pa[x],pa[x]=y;
12 }
13 void kruskal(){
14     ll cnt=0,wi=0;
15     while(!adj.empty() && cnt<n-1){
16         auto [w,u,v]=*adj.begin(); adj.erase(adj.begin());
17         if(find(u)!=find(v)) wi+=w,un(u,v);
18     }
19 }

```

binary-lifing.cpp

```

1  constexpr ll mxN=2e5+1;
2  ll n,q,p[mxN][31]={0};
3  vc<ll> adj[mxN];
4  void solve(istream &cin){
5      cin>>n>>q;
6      forn(i,2,n+1) cin>>p[i][0];
7      forn(j,1,31) forn(i,1,n+1) p[i][j]=p[p[i][j-1]][j-1];
8      while(q--){
9          ll x,k; cin>>x>>k;
10         forn(i,0,30) if(k&(1<<i)) x=p[x][i];
11         cout << (x?x:-1) << '\n';
12     }
13 }

```

find-centriod.cpp

```

1  void dfs(ll u=1,ll p=0){
2      d[u]=1;
3      for(auto&v:adj[u]){
4          if(v==p)
5              continue;
6          dfs(v,u);
7          d[u]+=d[v];
8      }
9  }
10 ll dfs_solve(ll u=1,ll p=0){
11     for(auto&v:adj[u]){
12         if(v==p)
13             continue;
14         if(d[v]*2>n) return dfs_solve(v,u);
15     }
16     return u;
17 }

```

hld.cpp

```

1  constexpr ll mxN=2e5+1;
2  ll tin=0,in[mxN]{},t[mxN<<1];
3  ll n,q,a[mxN],d[mxN]{},e[mxN]{};
4  vc<ll> adj[mxN];
5  ll dfs(ll u=1,ll p=0){
6      ll mx=0,sub=1;
7      forn(i,0,adj[u].size()){
8          ll v=adj[u][i];
9          if(v!=p){d[v]=d[u]+1; ll val=dfs(v,u);
10             ↪ sub+=val; if(val>mx) mx=val,e[u]=v;}
11             else swap(adj[u][i],adj[u][0]);
12         }
13     return sub;
14 }
15 void hld(ll u=1,ll p=1){
16     in[u]=++tin,t[in[u]+n-1]=a[u];
17     if(e[u]) hld(e[u],p);
18     forn(i,1,adj[u].size()){ll v=adj[u][i];
19         ↪ if(e[u]!=v) hld(v,v);}
20     e[u]=p;
21 }
22 void pst(ll p,ll val){for(t[p+=n-1]=val;p>=1;t[p]=max(t[p],
23     ↪ <<1,t[p<<1|1]));}
24 ll qry(ll l,ll r){
25     ll ans=0;
26     for(l+=n-1,r+=n-1;l<=r;l>=1,r>=1){if(l&1
27         ↪ ans=max(ans,t[l++]); if(r&1~1
28         ↪ ans=max(ans,t[r--]);}
29     return ans;
30 }
31 ll query(ll a,ll b){
32     ll ans=0;
33     for(;e[a]!=e[b];b=adj[e[b]][0]){
34         if(d[e[a]]>d[e[b]]) swap(a,b);
35         ans=max(ans,qry(in[e[b]],in[b]));
36     }
37 }

```

```

31     }
32     if(in[a]>in[b]) swap(a,b);
33     return max(ans,qry(in[a],in[b]));
34 }
35 void solve(istream &cin){
36     cin>>n>>q;
37     forn(i,1,n+1) cin>>a[i];
38     forn(i,0,n-1){ll u,v; cin>>u>>v;
39         ↪ adj[u].emp(v),adj[v].emp(u);}
40     adj[1].emp(0);
41     dfs();
42     hld();
43     terr;
44     forr(i,n-1,1) t[i]=max(t[i<<1],t[i<<1|1]);
45     for(ll op,a,b;q--){
46         cin>>op>>a>>b;
47         if(op==1) pst(in[a],b);
48         else cout << query(a,b) << ' ';
49     }
50 }

```

small-to-large.cpp

```

1  constexpr ll mxN=2e5+1;
2  ll n,k,res=0;
3  vc<ll> adj[mxN];
4  deque<ll> dfs(ll u=1,ll p=0){
5      deque<ll> cur(1,{1}),tmp;
6      for(auto &v:adj[u]){
7          if(v!=p){
8              tmp=dfs(v,u),tmp.push_front(0);
9              if(tmp.size()>cur.size())
10                 ↪ swap(tmp,cur);
11              forn(i,0,tmp.size()) if(k-i>=0 &&
12                 ↪ k-i<cur.size())
13                 ↪ res+=cur[k-i]*tmp[i];
14              forn(i,0,tmp.size())
15                 ↪ cur[i]+=tmp[i];
16          }
17      }
18      return cur;
19 }
20 void solve(istream &cin){
21     cin>>n>>k;
22     forn(i,0,n-1){ll u,v; cin>>u>>v;
23         ↪ adj[u].emp(v),adj[v].emp(u);}
24     dfs();
25     cout << res << '\n';
26 }

```

tree-diameter.cpp

```

1  #define emp emplace_back()
2  ll n,res=0;
3  vc<ll> adj[mxN],dp(mxN);
4
5  void dfs(ll u=1,ll p=0){dp[u]=0; for(auto&v:adj[u])
6      ↪ if(v!=p) dfs(v,u),res=max(res,dp[u]+dp[v]+1),dp[u]=max
7      ↪ (dp[u],dp[v]+1);}
8
9  int main()
10 {
11     cin>>n; forn(i,0,n-1){ll u,v; cin>>u>>v;
12         ↪ adj[u].emp(v),adj[v].emp(u);}
13     dfs();
14     cout << res << '\n';
15 }

```

3 Graph

2-sat.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll sc=0,scc[mxN];
3 ll id=0,st[mxN],vs[mxN]{};
4 ll n,m,tin=0,dfn[mxN]{};low[mxN];
5 vc<ll> adj[mxN];
6 void tarjan(ll u){
7     dfn[u]=low[u]=++tin,st[++id]=u,vs[u]=1;
8     for(auto &v:adj[u]){
9         if(!dfn[v])
10            ↪ tarjan(v),low[u]=min(low[u],low[v]);
11         else if(vs[v]) low[u]=min(low[u],dfn[v]);
12     }
13     if(low[u]==dfn[u]){
14         sc++;
15         for(;id;){
16             scc[st[id]]=sc,vs[st[id]]=0;
17             id--;
18             if(st[id+1]==u) break;
19         }
20     }
21 void solve(istream &cin){
22     cin >> n >> m;
23     forn(i,1,n+1){
24         char c1,c2; ll x,y,X,Y; cin >> c1 >> x >>
25         ↪ c2 >> y;
26         X=x+m,Y=y+m;
27         if(c1=='-') swap(x,X);
28         if(c2=='-') swap(y,Y);
29         adj[X].emp(y),adj[Y].emp(x);
30     }
31     forn(i,1,2*m+1) if(!dfn[i]) tarjan(i);
32     forn(i,1,m+1) if(scc[i]==scc[i+m]){cout <<
33         ↪ "IMPOSSIBLE" << '\n'; return;}
34     forn(i,1,m+1) cout << (scc[i]>scc[i+m]?'-':'+') <<
35         ↪ " \n"[i==m];
36 }

```

BellmanFord.cpp

```

1 //SFPA
2
3 bool sfpa(){
4     vc<ll> vs(n,0);
5     queue<ll> q; q.emplace(start);
6     while(!q.empty()){
7         ll u=q.front(); q.pop(),vs[u]=0;
8         for(auto&[v,w]:adj[u]){
9             if(d[v]>d[u]+w){
10                 d[v]=d[u]+w;
11                 if(!vs[v]){vs[v]=1,cnt[v]++; if(cnt[v]>n)
12                     ↪ return false; q.emplace(v);}
13             }
14         }
15     }
16     return true;

```

Dijkstra.cpp

```

1 void djc(){
2     mls<pll> hp;
3     res[1]=0,hp.emplace(pll{res[1],1});
4     while(!hp.empty()){
5         auto [d,u]=*hp.bg(); hp.erase(hp.bg());
6         if(d>res[u]) continue;
7         for(auto&[v,w]:adj[u]){
8             if(w+res[u]<res[v]) res[v]=w+res[u],
9             ↪ hp.emplace(pll{res[v],v});

```

```

    }
}

```

Edmond.cpp

```

1 constexpr ll mxN=505;
2 ll n,m,adj[mxN][mxN]{};g[mxN][mxN]{}; //adj=capacity
3 vc<ll> pa;
4 ll bfs(){
5     queue<pll> q; q.emplace(pll{0,inf});
6     while(!q.empty()){
7         auto [u,flow]=q.front(); q.pop();
8         forn(i,0,n)
9             if(g[u][i] && adj[u][i] &&
10                ↪ pa[i]==-1){
11                 pa[i]=u;
12                 if(i==n-1) return
13                 ↪ min(flow,adj[u][i]);
14                 q.emplace(pll{i,min(flow,a_j
15                 ↪ dj[u][i])});
16             }
17     }
18     return 0;
19 }
20 void solve(istream &cin){
21     cin>>n>>m;
22     pa.resize(n);
23     forn(i,1,m+1){ll u,v; cin>>u>>v,u--,v--;
24         ↪ adj[u][v]=g[u][v]=1,adj[v][u]=g[v][u]=1;}
25     ll f=0;
26     fill(all(pa),-1),pa[0]=0;
27     for(ll nf;nf=bfs();f+=nf){
28         ll cur=n-1;
29         while(cur) adj[pa[cur]][cur]=nf,adj[cur][
30         ↪ pa[cur]]+=nf,cur=pa[cur];
31         fill(all(pa),-1),pa[0]=0;
32     }
33     vc<pll> res;
34     forn(i,0,n) forn(j,0,n) if(g[i][j] && pa[i]!=-1 &&
35         ↪ pa[j]==-1) res.emplace(pll{i,j});
36     cout << res.size() << '\n';
37     for(auto&[u,v]:res) cout << u+1 << ' ' << v+1 <<
38         ↪ '\n';
39 }
40 int main()
41 {
42     fast_io;
43     ll testcase;
44     // cin>>testcase;
45     testcase=1;
46     while(testcase--){
47         solve(cin);
48         return 0;
49     }

```

articulation-point.cpp

```

1 //undirected graph articulation point
2 ll n,tim=0,art=0;
3 vc<ll> dfn(mxN,0),low(mxN);
4 void tarjan(ll u,ll p){
5     ll cnt=0;
6     dfn[u]=low[u]=++tim;
7     for(auto&v:adj[u]){
8         if(!dfn[v]){
9             tarjan(v,u);
10            low[u]=min(low[u],low[v]);
11            if(low[v]>dfn[u] && u!=root){ //root=dfn[1]
12                ++art;
13            }
14            cnt++;

```

```

15     }
16     else low[u]=min(low[u],dfn[v]);
17 }
18 if(u==root && cnt>1) art++;
19 }

```

bridge.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,dfn[mxN]={},low[mxN],tin=0;
3 vc<ll> adj[mxN];
4 vc<ar<ll,2>> res;
5 void tarjan(ll u=1,ll p=0){
6     dfn[u]=low[u]=++tin;
7     for(auto &v:adj[u]){
8         if(v==p) continue;
9         if(dfn[v]) low[u]=min(low[u],dfn[v]);
10        else{
11            tarjan(v,u),low[u]=min(low[u],low[v]);
12            if(low[v]>dfn[u])
13                res.emp(ar<ll,2>{u,v});
14        }
15 }

```

dinic.cpp

```

1 constexpr ll mxN=505;
2 class fe{
3 public:
4     ll u,v,cp,fw;
5     fe(){}
6     fe(ll U,ll V,ll CP):u(U),v(V),cp(CP){fw=0;}
7 };
8 ll n,m,sz=0;
9 vc<fe> edg;
10 queue<ll> q;
11 vc<ll> adj[mxN],ptr,dep;
12 bool bfs(){
13     while(!q.empty()){
14         ll u=q.front(); q.pop();
15         for(auto&id:adj[u]){
16             if(edg[id].cp==edg[id].fw ||
17                dep[edg[id].v]==-1) continue;
18             dep[edg[id].v]=dep[u]+1,q.emplace(id);
19         }
20     }
21     return dep[n-1]!=-1;
22 }
23 ll dinic(ll u,ll push){
24     if(!push) return 0;
25     if(u==n-1) return push;
26     for(ll &cid=ptr[u];cid<adj[u].size();++cid){
27         ll id=adj[u][cid];
28         ll v=edg[id].v;
29         if(dep[v]!=dep[u]+1) continue;
30         ll tr=dinic(v,min(push,edg[id].cp-edg[id].fw));
31         if(!tr) continue;
32         edg[id].fw+=tr,edg[id^1].fw-=tr;
33         return tr;
34     }
35     return 0;
36 }
37 void solve(istream &cin){
38     cin>>n>>m;
39     ptr.resize(n),dep.resize(n);
40     forn(i,1,m+1){
41         ll u,v,c; cin>>u>>v>>c,u--,v--;
42         edg.emp(fe{u,v,c}),edg.emp(fe{v,u,0});
43         adj[u].emp(sz),adj[v].emp(sz+1),sz+=2;

```

```

44     }
45     ll res=0;
46     for(;;){
47         ll flow;
48         fill(all(dep),-1);
49         q.emplace(0),dep[0]=0;
50         if(!bfs()) break;
51         fill(all(ptr),0);
52         for(;flow=dinic(0,inf);res+=flow);
53     }
54     cout << res << '\n';
55 }

```

```

56 int main()
57 {
58     fast_io;
59     ll testcase;
60     // cin>>testcase;
61     testcase=1;
62     while(testcase--){
63         solve(cin);
64         return 0;
65 }

```

dominator-tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,tim=0,id[mxN],sd[mxN],pa[mxN],f[mxN],lb[mxN],et[mxN];
3 vc<ll> adj[mxN],radj[mxN],t[mxN],st;
4 void dfs(ll u=1){
5     et[u]=++tim,ret[tim]=u;
6     id[tim]=sd[tim]=lb[tim]=pa[tim]=tim;
7     for(auto&v:adj[u]){
8         if(!et[v]) dfs(v),f[et[v]]=et[u];
9         radj[et[v]].emp(et[u]);
10    }
11 }
12 ll find(ll u,ll x=0){
13     if(u==pa[u]) return x?-1:u;
14     ll v=find(pa[u],x+1);
15     if(v<0) return u;
16     if(sd[lb[u]]>sd[lb[pa[u]]]) lb[u]=lb[pa[u]];
17     pa[u]=v;
18     return x?v:lb[u];
19 }
20 void built(){
21     vc<ll> arr[n+1];
22     forr(i,n,1){
23         for(auto&v:radj[i])
24             sd[i]=min(sd[i],sd[find(v)]);
25         if(i>1) arr[sd[i]].emp(i);
26         for(auto&v:arr[i]){
27             ll x=find(v);
28             if(sd[x]==sd[v]) id[v]=sd[v];
29             else id[v]=x;
30         }
31         if(i>1) pa[i]=f[i];
32     }
33     forn(i,2,n+1){
34         if(id[i]!=sd[i]) id[i]=id[id[i]];
35         t[ret[i]].emp(ret[id[i]]),t[ret[id[i]]].emp(i);
36         p[ret[i]];
37     }
38 }
39 void dfs_ans(ll u=1,ll p=0){
40     st.emp(u);
41     if(u==n){
42         sort(all(st));
43         cout << st.size() << '\n';
44         for(auto&v:st) cout << v << ' '; cout << '\n';
45         exit(0);
46     }
47     for(auto&v:t[u]){
48         if(v==p) continue;
49         dfs_ans(v,u);

```

```

48     }
49     st.pop_back();
50 }
51 void solve(istream &cin){
52     cin>>n>>m;
53     forn(i,1,m+1){
54         ll u,v; cin>>u>>v;
55         adj[u].emp(v);
56     }
57     dfs();
58     built();
59     dfs_ans();
60 }
61

```

floyd-warshall.cpp

```

1  for (k = 1; k <= n; k++) {
2      for (x = 1; x <= n; x++) {
3          for (y = 1; y <= n; y++) {
4              f[x][y] = min(f[x][y], f[x][k] + f[k][y]);
5          }
6      }
7  }

```

hierholzer.cpp

```

1  //debrujin sequence
2  ll n;
3  string s;
4  vc<ll> adj[1<<14|1];
5  void dfs(ll u=0){
6      while(!adj[u].empty()){
7          ll v; v=adj[u].back(); adj[u].pop_back();
8          dfs(v);
9          s+=(v&1)?'1':'0';
10     }
11 }
12 void solve(istream &cin){
13     cin>>n;
14     if(n==1){cout << "01" << '\n'; return;}
15     forn(i,0,1<<(n-1)){
16         adj[i].emp(((i<<1)&((1<<(n-1))-1)));
17         adj[i].emp(((i<<1|1)&((1<<(n-1))-1)));
18     }
19     dfs();
20     forn(i,0,n-1) s+='0';
21     reverse(all(s));
22     cout << s << '\n';
23 }

```

hopcroft-karp.cpp

```

1  constexpr ll mxN=501;
2  ll n,m,k,mat[mxN<<1]{},d[mxN<<1];
3  vc<ll> adj[mxN];
4  ll bfs(){
5      queue<ll> q;
6      memset(d,-1,sizeof(d));
7      forn(i,1,n+1) if(!mat[i]) d[i]=0,q.emplace(i);
8      while(!q.empty()){
9          ll u=q.front(); q.pop();
10         if(~d[0]) return 1;
11         for(auto &v:adj[u]){
12             if(!d[mat[v]]) d[mat[v]]=d[u]+1;
13             q.emplace(mat[v]);
14         }
15     }
16     return 0;
17 }
18 bool dfs(ll u){
19     if(!u) return true;

```

```

19     for(auto &v:adj[u]) if(d[mat[v]]==d[u]+1 &&
20         ↪ dfs(mat[v])){mat[u]=v,mat[v]=u;return true;}
21     d[u]=-1;
22     return false;
23 }
24 void solve(istream &cin){
25     cin >> n >> m >> k;
26     forn(i,0,k){ll u,v; cin >> u >> v;
27         ↪ adj[u].emp(n+v);}
28     ll res=0;
29     for(;bfs();) forn(i,1,n+1) if(!mat[i]) res+=dfs(i);
30     cout << res << '\n';
31     if(res) forn(i,1,n+1) if(mat[i]) cout << i << ' '
32         ↪ << (mat[i]-n) << '\n';
33 }

```

kuhn.cpp

```

1  constexpr ll mxN=1001;
2  constexpr ll base=500;
3  ll n,m,k,vs1[mxN],vs2[mxN],ok[mxN][mxN]{};
4  vc<ll> adj[mxN];
5  bool dfs(ll u){
6      if(vs2[u]) return false;
7      vs2[u]=1;
8      for(auto&v:adj[u]) if(!vs1[v] ||
9         ↪ dfs(vs1[v])){vs1[v]=u; return true;}
10     return false;
11 }
12 void solve(istream &cin){
13     cin >> n >> m >> k;
14     // [1,500] boy [500+1,500+500] girl
15     forn(i,0,k){
16         ll u,v; cin >> u >> v,v+=base;
17         if(!ok[u][v])
18             ↪ adj[u].emp(v),adj[v].emp(u),ok[u][v]=1;
19     }
20     memset(vs1,0,sizeof(vs1));
21     forn(i,1,n+m+1){
22         memset(vs2,0,sizeof(vs2));
23         dfs(i);
24     }
25     vc<ar<ll,2>>> res;
26     forn(i,1,n+1){
27         for(auto &v:adj[i]){
28             if(vs1[v]==i)
29                 ↪ res.emp(ar<ll,2>>>{i,v-base});
30         }
31     }
32     cout << res.size() << '\n';
33     for(auto&[u,v]:res) cout << u << ' ' << v << '\n';
34 }

```

mcmf.cpp

```

1  class Edge{
2      public:
3          ll u,v,c,w;
4          Edge(){}
5          Edge(ll u,ll v,ll c,ll
6             ↪ w):u(u),v(v),c(c),w(w){}
7  };
8  constexpr ll mxN=501;
9  ll n,m,k,id=0,vs[mxN]{},d[mxN],pa[mxN];
10 vc<Edge> e;
11 vc<ll> adj[mxN];
12 ll bfs(){
13     memset(pa,-1,8*(n+1)),memset(d,0x3f,sizeof(d));
14     queue<ll> q;
15     d[1]=0,vs[1]=1,q.emplace(1);
16     while(!q.empty()){
17         auto u=q.front(); q.pop(),vs[u]=0;

```

```

17         for(auto &eid:adj[u]){
18             auto [x,v,c,w]=e[eid];
19             if(c && d[u]+w<d[v]){
20                 pa[v]=eid,d[v]=d[u]+w;
21                 if(!vs[v])
22                     vs[v]=1,q.emplace(v);
23             }
24         }
25         return d[n]<INF;
26     }
27     void solve(){
28         cin >> n >> m >> k;
29         forn(i,1,m+1){
30             ll u,v,c,w; cin >> u >> v >> c >> w;
31             e.emp(Edge{u,v,c,w}),e.emp(Edge{v,u,0,-w});
32             adj[u].emp(id++),adj[v].emp(id++);
33         }
34         ll f=0,res=0;
35         for(ll nf,v;f<k && bfs();){
36             for(nf=k-f,v=n;v!=1;nf=min(nf,e[pa[v]].c),
37                 v=e[pa[v]].u);
38             for(v=n;v!=1;e[pa[v]].c-=nf,e[pa[v]^1].c+=nf,
39                 nf,v=e[pa[v]].u);
40             res+=d[n]*nf,f+=nf;
41         }
42         cout << (f<k?-1:res) << '\n';
43     }
44 }

```

scc.cpp

```

1 void dfs(ll u){
2     dfn[u]=low[u]=++tim,st[++id]=u,vs[u]=1;
3     for(auto &v:adj[u]){
4         if(!dfn[v])
5             dfs(v),low[u]=min(low[u],low[v]);
6         else if(vs[v]) low[u]=min(low[u],dfn[v]);
7     }
8     if(low[u]==dfn[u]){
9         for(ll v=-1;v!=u;scc[st[id]]=sc,vs[st[id]]=1,
10             id--,v=st[id],id--);
11         sc++;
12     }
13 }
14 }

```

toposort.cpp

```

1 void toposort() {
2     int n, m;
3     cin >> n >> m;
4     vector<vector<int>> graph(n+1);
5     for (int i = 0; i < m; i++) {
6         int a, b;
7         cin >> a >> b;
8         graph[a].push_back(b);
9     }
10    vector<int> rudu(n+1, 0);
11    for (auto &node : graph) {
12        for (auto &now : node) {
13            rudu[now]++;
14        }
15    }
16    queue<int> que;
17    for (int i = 1; i <= n; i++) {
18        if (rudu[i] == 0) que.push(i);
19    }
20    vector<int> topo;
21    while (!que.empty()) {
22        int top = que.front();
23        que.pop();
24        topo.push_back(top);
25        for (auto &i : graph[top]) {
26            rudu[i]--;
27            if (rudu[i] == 0) que.push(i);
28        }
29    }
30 }

```

```

28     }
29 }
30 if (topo.size() == n) {
31     for (auto &i : topo) cout << i << " ";
32 }
33 else
34     cout << "IMPOSSIBLE";
35 }

```

4 Geometry

Determine whether a polygon is convex.cpp

```

1 #include <vector>
2 using namespace std;
3
4 struct Point {
5     double x, y;
6 };
7
8 double cross(const Point& a, const Point& b, const Point&
9     c) {
10     // 計算向量 ab x bc
11     double x1 = b.x - a.x, y1 = b.y - a.y;
12     double x2 = c.x - b.x, y2 = c.y - b.y;
13     return x1 * y2 - y1 * x2;
14 }
15
16 bool isConvex(const vector<Point>& poly) {
17     int n = poly.size();
18     if (n < 3) return false;
19     int sign = 0;
20     for (int i = 0; i < n; ++i) {
21         double c = cross(poly[i], poly[(i + 1) % n],
22             poly[(i + 2) % n]);
23         if (c != 0) {
24             if (sign == 0)
25                 sign = (c > 0) ? 1 : -1;
26             else if ((c > 0 && sign < 0) || (c < 0 && sign
27                 > 0))
28                 return false;
29         }
30     }
31     return true;
32 }

```

Rotating Calipers 求凸包直徑.cpp

```

1 struct Point {
2     double x, y;
3 };
4
5 double dist2(const Point& a, const Point& b) {
6     double dx = a.x - b.x, dy = a.y - b.y;
7     return dx * dx + dy * dy;
8 }
9
10 // 假設 convex 是已經逆時針排序的凸包
11 double convexHullDiameter(const vector<Point>& convex) {
12     int n = convex.size();
13     if (n == 1) return 0;
14     if (n == 2) return dist2(convex[0], convex[1]);
15     double res = 0;
16     for (int i = 0, j = 1; i < n; ++i) {
17         while (dist2(convex[i], convex[(j + 1) % n]) >
18             dist2(convex[i], convex[j]))
19             j = (j + 1) % n;
20         res = max(res, dist2(convex[i], convex[j]));
21     }
22     return sqrt(res);
23 }

```


all solutions for Diophantine Equation in given range.cpp

```

1  /*
2      1. General solution to the linear Diophantine equation:
3           $a * x + b * y = c$ 
4          If  $(x_0, y_0)$  is a particular solution and  $g = \gcd(a, b)$ :
5               $x = x_0 + k * (b / g)$ 
6               $y = y_0 - k * (a / g)$ 
7              for any integer  $k$ 
8      2. Number of integer solutions  $(x, y)$  within bounds:
9           $minx \leq x \leq maxx$ 
10          $miny \leq y \leq maxy$ 
11         If  $lx$  and  $rx$  are the valid range for  $x$ :
12             Number of solutions =  $(rx - lx) / \gcd(a, b) + 1$ 
13 */
14 void shift_solution(int & x, int & y, int a, int b, int
15     cnt) {
16     x += cnt * b;
17     y -= cnt * a;
18 }
19 int find_all_solutions(int a, int b, int c, int minx, int
20     maxx, int miny, int maxy) {
21     int x, y, g;
22     if (!find_any_solution(a, b, c, x, y, g))
23         return 0;
24     a /= g;
25     b /= g;
26     int sign_a = a > 0 ? +1 : -1;
27     int sign_b = b > 0 ? +1 : -1;
28
29     shift_solution(x, y, a, b, (minx - x) / b);
30     if (x < minx)
31         shift_solution(x, y, a, b, sign_b);
32     if (x > maxx)
33         return 0;
34     int lx1 = x;
35
36     shift_solution(x, y, a, b, (maxx - x) / b);
37     if (x > maxx)
38         shift_solution(x, y, a, b, -sign_b);
39     int rx1 = x;
40
41     shift_solution(x, y, a, b, -(miny - y) / a);
42     if (y < miny)
43         shift_solution(x, y, a, b, -sign_a);
44     if (y > maxy)
45         return 0;
46     int lx2 = x;
47
48     shift_solution(x, y, a, b, -(maxy - y) / a);
49     if (y > maxy)
50         shift_solution(x, y, a, b, sign_a);
51     int rx2 = x;
52
53     if (lx2 > rx2)
54         swap(lx2, rx2);
55     int lx = max(lx1, lx2);
56     int rx = min(rx1, rx2);
57
58     if (lx > rx)
59         return 0;
60     return (rx - lx) / abs(b) + 1;
61 }

```

convex-hull.cpp

```

1  class Point{
2      public:
3          ll x,y;
4          Point(){ }
5          Point(ll x,ll y):x(x),y(y){ }

```

```

friend ll operator^(const Point a,const
    Point b){return a.x*b.y-a.y*b.x;}
friend ll operator*(const Point a,const
    Point b){return a.x*b.x+a.y*b.y;}
friend Point operator-(const Point a,const
    Point b){
    return Point{a.x-b.x,a.y-b.y};
}
friend ostream& operator>>(ostream
    & in,Point &a){
    in >> a.x >> a.y;
    return in;
}
friend ostream& operator<<(ostream
    & out,Point &a){
    out << a.x << ' ' << a.y << '\n';
    return out;
}
friend bool operator<(const Point a,const
    Point b){return a.x<b.x||(a.x==b.x &&
    a.y<b.y);}
};
ll dis(Point a,Point b){
    return (a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y);
}
ll ok(Point a,Point b,Point c){
    ll ang=(b-a)^(c-a); //ang>0 false ang< true
    return ang<0;
}
constexpr ll mxN=2e5+5;
ll st[mxN],id=0;
void solve(istream &cin){
    ll n; cin >> n;
    vc<Point> a(n); for(auto &v:a) cin >> v;
    forn(i,1,n) if(a[i]<a[0]) swap(a[i],a[0]);
    sort(1+all(a),[p=a[0]](Point a,Point b){ll
        ang=(a-p)^(b-p); return
        !ang?dis(p,a)<dis(p,b):ang>0;});
    {
        ll j; for(j=n-1;j>=0 &&
            ((a[j]-a[0])^(a[n-1]-a[0]))==0;j--);
        reverse(j+1+a.bg(),a.ed());
    }
    forn(i,0,n){
        while(id>1 &&
            ok(a[st[id-1]],a[st[id]],a[i])) --id;
        st[++id]=i;
    }
    cout << id << '\n';
    if(id) for(;id;cout << a[st[id--]]);
}

```

cross-product.cpp

```

1  class Point{
2      public:
3          ll x,y;
4          Point(){ }
5          Point(ll x,ll y):x(x),y(y){ }
6          ll f(Point a,Point b){
7              return a.x*b.y-b.x*a.y;
8          } //cross product
9          friend Point operator-(const Point a,const
10             Point b){
11             return Point{a.x-b.x,a.y-b.y};
12         }
13         friend ostream& operator>>(ostream
14             & in,Point &a){
15             in >> a.x >> a.y;
16             return in;
17         }
18     };

```

extend-li-chao-tree.cpp

```

1 class Line{
2     public:
3         ll m,c;
4         Line(){m=0,c=-1;}
5         Line(ll m,ll c):m(m),c(c){}
6         ll cal(ll x){return m*x+c;}
7 };
8 constexpr ll mxN=1e5;
9 Line t[mxN<<2];
10 void insert(ll i,ll sl,ll sr,ll l,ll r,Line L){
11     if(l>sr || r<sl) return ;
12     ll mid=(sl+sr)>>1;
13     if(l<=sl && sr<=r){
14         if(L.cal(mid)>t[i].cal(mid)) swap(L,t[i]);
15         if(sl==sr) return;
16         if(L.m>t[i].m)
17             ↪ insert(i<<1|1,mid+1,sr,l,r,L);
18         else insert(i<<1,sl,mid,l,r,L);
19         return ;
20     }
21     insert(i<<1,sl,mid,l,r,L);
22     insert(i<<1|1,mid+1,sr,l,r,L);
23 }
24 ll query(ll i,ll sl,ll sr,ll x){
25     ll mid=(sl+sr)>>1;
26     if(sl==sr) return t[i].cal(x);
27     if(x<=mid) return
28         ↪ max(t[i].cal(x),query(i<<1,sl,mid,x));
29     else return
30         ↪ max(t[i].cal(x),query(i<<1|1,mid+1,sr,x));
31 }
32 void solve(istream &cin){
33     ll n,m,r0=-1; cin >> n >> m;
34     forn(i,0,n){
35         ll x1,x2,y1,y2; cin >> x1 >> y1 >> x2 >>
36         ↪ y2;
37         if(!x1) r0=max(r0,y1);
38         Line l((y2-y1)/(x2-x1),-x1*(y2-y1)/(x2-x1)+y1);
39         insert(1,1,mxN,x1,x2,1);
40     }
41     cout << r0 << " \n"[m==1];
42     forn(i,1,m+1) cout << query(1,1,mxN,i) << "
43         ↪ \n"[i==m];
44 }

```

li-chao-tree.cpp

```

1 class Line{
2     public:
3         ll m,c;
4         Line(){m=c=0;}
5         Line(ll m,ll c):m(m),c(c){}
6         ll cal(ll x){return m*x+c;}
7 };
8 constexpr ll mxN=1e5;
9 Line t[mxN<<2];
10 void insert(ll i,ll sl,ll sr,Line l){
11     ll mid=(sl+sr)>>1;
12     if(l.cal(mid)>t[i].cal(mid)) swap(l,t[i]);
13     if(sl==sr) return;
14     if(l.m>t[i].m) insert(i<<1|1,mid+1,sr,l);
15     else insert(i<<1,sl,mid,l);
16 }
17 ll query(ll i,ll sl,ll sr,ll x){
18     ll mid=(sl+sr)>>1;
19     if(sl==sr) return t[i].cal(x);
20     if(x<=mid) return
21         ↪ max(t[i].cal(x),query(i<<1,sl,mid,x));
22     else return
23         ↪ max(t[i].cal(x),query(i<<1|1,mid+1,sr,x));
24 }
25 void solve(istream &cin){
26     ll n,m; cin >> n >> m;

```

```

forn(i,0,n){
    ll y1,y2; cin >> y1 >> y2;
    // (y2-y1)/(m-0) = (y-y1)/x
    //x * (y2-y1)/m + y1= y
    Line l((y2-y1)/m,y1);
    insert(1,1,mxN,1);
}
forn(i,0,m+1) cout << query(1,1,mxN,i) << "
↪ \n"[i==m];
}

```

max-man-dis.cpp

```

constexpr ll mxN=2e5+1;
ll n,d[2][2];
void solve(){
    cin >> n;
    d[0][0]=d[1][0]=-inf;
    d[0][1]=d[1][1]=inf;
    forn(i,0,n){
        ll x,y;
        cin >> x >> y;
        d[0][0]=max(d[0][0],x+y);
        d[0][1]=min(d[0][1],x+y);
        d[1][0]=max(d[1][0],x-y);
        d[1][1]=min(d[1][1],x-y);
        cout <<
        ↪ max(d[0][0]-d[0][1],d[1][0]-d[1][1])
        ↪ << '\n';
    }
}

```

minimum-euc-dis.cpp

```

class Point{
    public:
        ll x,y;
        Point(){x(0),y(0)}
        Point(ll x,ll y):x(x),y(y){}
        friend ll operator^(const Point a,const
        ↪ Point b){return a.x*b.y-a.y*b.x;}
        friend ll operator*(const Point a,const
        ↪ Point b){return a.x*b.x+a.y*b.y;}
        friend Point operator-(const Point a,const
        ↪ Point b){
            return Point{a.x-b.x,a.y-b.y};
        }
        friend istream& operator>>(istream
        ↪ &in,Point &a){
            in >> a.x >> a.y;
            return in;
        }
        friend bool operator<(const Point a,const
        ↪ Point b){return a.x<b.x||(a.x==b.x &&
        ↪ a.y<b.y);}
};
ll euc(Point a,Point b){
    return (a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y);
}
void solve(istream &cin){
    ll n; cin >> n;
    vc<Point> a(n); for(auto &v:a) cin >> v;
    sort(all(a),[](Point u,Point v){return
    ↪ (u.y<v.y)||(u.y==v.y && u.x<v.x)});
    ll d;
    mls<Point> hp;
    d=euc(a[0],a[1]),hp.emplace(a[0]),hp.emplace(a[1]);
    for(ll i=2,p=0;i<n;i++){
        for(;p<n && (a[p].y-a[i].y)*(a[p].y-a[i].y)
        ↪ >=d;hp.erase(hp.lwb(a[p])),p++);
        auto it=hp.lwb(a[i]);
        auto front=it,back=it;
        forn(j,0,4){

```

```

32         if(front!=hp.ed())
33             ↪ d=min(d,euc(a[i],*front));
34         else break; front++;
35     }
36     forn(j,0,4){
37         if(back!=hp.ed())
38             ↪ d=min(d,euc(a[i],*back));
39         {if(back!=hp.bg()) back--; else
40             ↪ break;}
41     }
42     hp.emplace(a[i]);
43     cout << d << '\n';
44 }

```

pick-theorem.cpp

```

1 void solve(istream &cin){
2     ll n; cin >> n;
3     vc<Point> a(n);
4     for(auto &v:a) cin >> v;
5     ll area=0,B=0;
6     forn(i,0,n){
7         ll j=(i+1)%n;
8         area+=a[i]^a[j];
9         B+=gcd(abs(a[i].x-a[j].x),abs(a[i].y-a[j].y));
10    }
11    cout << (abs(area)/2+1-B/2) << ' ' << B << '\n';
12    //area=I+B/2-1 I=internal node B=boundary node
13 }

```

ray-casting.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         friend ll operator^(const Point a,const
7             ↪ Point b){return a.x*b.y-a.y*b.x;}
8         friend ll operator*(const Point a,const
9             ↪ Point b){return a.x*b.x+a.y*b.y;}
10        friend Point operator-(const Point a,const
11            ↪ Point b){
12            return Point(a.x-b.x,a.y-b.y);
13        }
14        friend istream& operator>>(istream
15            ↪ &in,Point &a){
16            in >> a.x >> a.y;
17            return in;
18        }
19    };
20    bool on(Point &a,Point &b,Point &c){
21        if((b-a)^(c-a)) return false;
22        return 0<=((b-a)*(c-a)) &&
23            ↪ ((b-a)*(c-a))<=((b-a)*(b-a));
24    }
25    void query(vc<Point> &a,Point &p){
26        ll n=a.size(),ok=0;
27        forn(i,0,n){
28            ll j=(i+1)%n;
29            if(on(a[i],a[j],p)){cout << "BOUNDARY" <<
30                ↪ '\n'; return;}
31            if(p.y>a[i].y!=p.y>a[j].y){
32                ll sg=(a[j]-a[i])^(p-a[i]);
33                if(sg<0==a[i].y>a[j].y) ok^=1;
34            }
35        }
36        cout << (ok&1?"INSIDE":"OUTSIDE") << '\n';
37    }

```

動態凸包.cpp

```

1 struct Point {
2     long long x, y;
3     bool operator<(const Point& p) const {
4         return x < p.x || (x == p.x && y < p.y);
5     }
6 };
7
8 long long cross(const Point& a, const Point& b, const
9     ↪ Point& c) {
10    // 向量 ab x bc
11    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x
12        ↪ - a.x);
13 }
14
15 struct DynamicConvexHull {
16     set<Point> up, dn; // 上凸包、下凸包
17
18     void insert(const Point& p) {
19         insertHull(up, p, false); // 上凸包
20         insertHull(dn, p, true); // 下凸包
21     }
22
23     // 取得逆時針順序的凸包點 (不重複端點)
24     vector<Point> getConvexHull() const {
25         vector<Point> res;
26         for (auto p : up) res.push_back(p);
27         vector<Point> dnvec(dn.rbegin(), dn.rend());
28         for (int i = 1; i + 1 < dnvec.size(); ++i) //
29             ↪ 去掉首尾重複
30             res.push_back(dnvec[i]);
31         return res;
32     }
33
34 private:
35     // flag: false= 上凸包, true= 下凸包
36     void insertHull(set<Point>& hull, const Point& p, bool
37         ↪ flag) {
38         auto it = hull.insert(p).first;
39         // 刪除左側不在凸包上的點
40         while (it != hull.begin()) {
41             auto prv = prev(it);
42             if (prv == hull.begin()) break;
43             auto pprv = prev(prv);
44             if ((cross(*pprv, *prv, *it) > 0) ^ flag)
45                 hull.erase(prv);
46             else break;
47         }
48         // 刪除右側不在凸包上的點
49         while (next(it) != hull.end()) {
50             auto nxt = next(it);
51             if (next(nxt) == hull.end()) break;
52             auto nnxt = next(nxt);
53             if ((cross(*it, *nxt, *nnxt) > 0) ^ flag)
54                 hull.erase(nxt);
55             else break;
56         }
57     }
58 }

```

多邊形重心計算.cpp

```

1 struct Point {
2     double x, y;
3 };
4
5 Point polygonCentroid(const vector<Point>& poly) {
6     double cx = 0, cy = 0, area = 0;
7     int n = poly.size();
8     for (int i = 0; i < n; ++i) {
9         double cross = poly[i].x * poly[(i + 1) % n].y -
10             ↪ poly[(i + 1) % n].x * poly[i].y;
11         cx += (poly[i].x + poly[(i + 1) % n].x) * cross;
12         cy += (poly[i].y + poly[(i + 1) % n].y) * cross;
13     }
14 }

```

```

12     area += cross;
13 }
14 area /= 2.0;
15 cx /= (6.0 * area);
16 cy /= (6.0 * area);
17 return {cx, cy};
18 }

```

找三點組成最大圓.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4  typedef pair<ll, ll> P;
5
6  ll cross(P a, P b) { return a.first * b.second - a.second
    ↪ * b.first; }
7  ll dist2(P a, P b) {
8      ll dx = a.first - b.first, dy = a.second - b.second;
9      return dx * dx + dy * dy;
10 }
11 double dist(P a, P b) { return sqrt(dist2(a, b)); }
12
13 // Andrew's Monotone Chain
14 vector<P> convex_hull(vector<P> pts) {
15     sort(pts.begin(), pts.end());
16     int n = pts.size(), k = 0;
17     vector<P> hull(2 * n);
18     for (int i = 0; i < n; ++i) {
19         while (k >= 2 &&
20             ↪ cross({hull[k-1].first-hull[k-2].first,
21             ↪ hull[k-1].second-hull[k-2].second},
22                 {pts[i].first-hull[k-1].first,
23                 ↪ st,
24                 ↪ pts[i].second-hull[k-1].second}
25                 ↪ .second}) <=
26                 ↪ 0)
27             --k;
28         hull[k++] = pts[i];
29     }
30     hull.resize(k-1);
31     return hull;
32 }
33
34 double circumradius(P a, P b, P c) {
35     double A = dist(b, c), B = dist(a, c), C = dist(a, b);
36     double S = abs(cross({b.first-a.first,
37         ↪ b.second-a.second},
38         ↪ {c.first-a.first,
39         ↪ c.second-a.second})) / 2.0;
40
41     if (S == 0) return 0; // 共線
42     return (A * B * C) / (4.0 * S);
43 }
44
45 int main() {
46     int n;
47     cin >> n;
48     vector<P> pts(n);
49     for (int i = 0; i < n; ++i) cin >> pts[i].first >>
50         ↪ pts[i].second;
51     vector<P> hull = convex_hull(pts);
52     int m = hull.size();
53     double ans = 0;
54     // 旋轉卡殼
55     for (int i = 0; i < m; ++i) {

```

```

56         int j = (i + 1) % m, k = (j + 1) % m;
57         do {
58             while (true) {
59                 int k2 = (k + 1) % m;
60                 double r1 = circumradius(hull[i], hull[j],
61                     ↪ hull[k]);
62                 double r2 = circumradius(hull[i], hull[j],
63                     ↪ hull[k2]);
64                 if (r2 > r1) k = k2;
65                 else break;
66             }
67             ans = max(ans, circumradius(hull[i], hull[j],
68                 ↪ hull[k]));
69             j = (j + 1) % m;
70         } while (j != i);
71     }
72     cout << fixed << setprecision(6) << ans << endl;
73 }

```

極角排序.cpp

```

1  struct Point {
2      int x, y;
3  };
4
5  // 以 base 為基準點，對 points 進行極角排序
6  void polarSort(vector<Point>& points, Point base) {
7      sort(points.begin(), points.end(), [&](const Point& a,
8         ↪ const Point& b) {
9          int dx1 = a.x - base.x, dy1 = a.y - base.y;
10         int dx2 = b.x - base.x, dy2 = b.y - base.y;
11         long long cross = 1LL * dx1 * dy2 - 1LL * dy1 *
12             ↪ dx2;
13         if (cross != 0) return cross > 0;
14         // 極角相同時，距離近的排前面
15         return dx1 * dx1 + dy1 * dy1 < dx2 * dx2 + dy2 *
16             ↪ dy2;
17     });
18 }

```

5 Data Structure

Array Version of Linked List.cpp

```

1  /* 看不懂 cut the node
2     實際用法需要理解 */
3  struct node{
4      struct node *nxt, *prv;
5      long long v;
6  }pos[100002];
7
8  inline node* cut(node *p) // cut the node
9  {
10     p->nxt->prv = p->prv;
11     p->prv->nxt = p->nxt;
12     p->nxt = p->prv = NULL;
13     return p;
14 }
15
16 inline node* insert(node *p, node *q) // insert node q to
17     ↪ right side of node p
18 {
19     q->nxt = p->nxt;
20     p->nxt->prv = q;
21     q->prv = p;
22     p->nxt = q;
23     return q;
24 }

```

Binary Index Tree.cpp

```

1 //BIT is always 1 index
2 ll n,t[mxN],a[mxN];
3 void built() {for(i,1,mxN) { t[i]=a[i]; if(i+(i&-i)<mxN)
4   ↳ t[i+(i&-i)]+=t[i];} } //O(n)
5 void pad(ll p,ll val) {for(;p<mxN;t[p]+=val,p+=p&-p);}
6 ll query(ll p) {ll ans=0; for(;p;ans+=t[p],p=p&-p); return
7   ↳ ans;}

```

Disjoint Set Union.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSets {
5     private:
6         vector<int> parents;
7         vector<int> sizes;
8
9     public:
10         DisjointSets(int size) : parents(size),
11   ↳ sizes(size, 1) {
12             for (int i = 0; i < size; i++) {
13   ↳ parents[i] = i; }
14         }
15
16         /** @return the "representative" node in x's
17   ↳ component */
18         int find(int x) {
19             return parents[x] == x ? x : (parents[x]
20   ↳ = find(parents[x]));
21         }
22
23         /** @return whether the merge changed connectivity
24   ↳ */
25         bool unite(int x, int y) {
26             int x_root = find(x);
27             int y_root = find(y);
28             if (x_root == y_root) { return false; }
29
30             if (sizes[x_root] < sizes[y_root]) {
31   ↳ swap(x_root, y_root); }
32             sizes[x_root] += sizes[y_root];
33             parents[y_root] = x_root;
34             return true;
35         }
36
37         /** @return whether x and y are in the same
38   ↳ connected component */
39         bool connected(int x, int y) { return find(x) ==
40   ↳ find(y); }
41     };

```

Segment Tree.cpp

```

1 //range add range query // sum query
2 #define forr(a,b,c) for(ll a=b;a>=c;--a)
3 #define form(a,b,c) for(ll a=b;a<=c;++a)
4 constexpr ll mxN=2e5+1;
5 #pragma GCC target("lzcnt")
6 ll n,q,t[mxN<<1],lz[mxN]{};
7 void built() {forr(i,n-1,1)t[i]=t[i<<1]+t[i<<1|1];}
8 void apply(ll p,ll k,ll val){t[p]+=val*k; if(p<n)
9   ↳ lz[p]+=val;}
10 void push(ll p){for(ll h=63-__builtin_clzll(p);h-->
11   ↳ if(lz[p>>h]){apply((p>>h)<<1,1<<(h-1),lz[p>>h]),apply(
12   ↳ (p>>h)<<1|1,1<<(h-1),lz[p>>h]),lz[p>>h]=0;}}
13 void built(ll p){for(;p>=1;)if(!lz[p])
14   ↳ t[p]=t[p<<1]+t[p<<1|1];}
15 void rad(ll l,ll r,ll val){
16     ll lo=l+n-1,r0=r+n-1,k=1; push(lo),push(r0);
17     for(;l<=r;l>=1,r>=1,k<=1){if(l&1) apply(l++,k,val);
18   ↳ if(r&1^1) apply(r--,k,val);}

```

```

    built(lo),built(r0);
}
ll query(ll l,ll r){ ll ans=0; push(l+=n-1),push(r+=n-1);
↳ for(;l<=r;l>=1,r>=1){if(l&1) ans+=t[l++]; if(r&1^1)
↳ ans+=t[r--];} return ans;} //range query
ll query(ll p){push(p+=n-1); return t[p];} // point query

```

persistence-seg-tree.cpp

```

1 constexpr ll mxN=2e5+1;
2 constexpr ll mxT=7e6+1;
3 ll n,q,t[mxT],a[mxN],lc[mxT],rc[mxT],rt[mxN],rsc,sz;
4 void built(ll i,ll sl,ll sr){
5     if(sl==sr){t[i]=a[sl]; return;}
6     ll mid; mid=(sl+sr)>>1;
7     lc[i]=++sz,rc[i]=++sz;
8     built(lc[i],sl,mid);
9     built(rc[i],mid+1,sr);
10    t[i]=t[lc[i]]+t[rc[i]];
11 }
12 ll pst(ll pos,ll val,ll i,ll sl,ll sr){
13     ll x; x=++sz;
14     if(sl==sr){t[x]=val; return x;}
15     ll mid; mid=(sl+sr)>>1;
16     if(pos<=mid)
17   ↳ lc[x]=pst(pos,val,lc[i],sl,mid),rc[x]=rc[i];
18     else lc[x]=lc[i],rc[x]=pst(pos,val,rc[i],mid+1,sr);
19     t[x]=t[lc[x]]+t[rc[x]];
20     return x;
21 }
22 ll query(ll l,ll r,ll i,ll sl,ll sr){
23     if(l>sr || r<sl) return 0;
24     if(l<=sl && sr<=r) return t[i];
25     ll mid; mid=(sl+sr)>>1;
26     return query(l,r,lc[i],sl,mid)+query(l,r,rc[i],mid
27   ↳ +1,sr);
28 }
29 void solve(istream &cin){
30     cin>>n>>q;
31     sz=rsc=0;
32     forn(i,1,n+1) cin>>a[i];
33     rt[++rsc]=++sz;
34     built(rt[1],1,n);
35     while(q--){
36         ll op; cin>>op;
37         if(op==3){
38             ll k; cin>>k;
39             rt[++rsc]=++sz;
40             lc[rt[rsc]]=lc[rt[k]],rc[rt[rsc]]=
41   ↳ rc[rt[k]],t[rt[rsc]]=t[lc[rt[r]]
42   ↳ +t[rc[rt[rsc]]];
43         }
44         else if(op==1){
45             ll k,a,x; cin>>k>>a>>x;
46             rt[k]=pst(a,x,rt[k],1,n);
47         }
48         else{
49             ll k,a,b; cin>>k>>a>>b;
50             cout << query(a,b,rt[k],1,n) <<
51   ↳ '\n';
52         }
53     }
54 }

```

splay-tree.cpp

```

1 constexpr ll mxN=2e5+3;
2 ll n,m,rt,t[mxN][2]{},cnt[mxN],val[mxN],f[mxN],lz[mxN]{};v
3   ↳ als[mxN];
4 string s;
5 ll dir(ll x){return t[f[x]][1]==x;}
6 void push_down(ll x){

```

```

6         if(lz[x]){
7             swap(t[x][0],t[x][1]);
8             if(t[x][0]) lz[t[x][0]]^=1;
9             if(t[x][1]) lz[t[x][1]]^=1;
10            lz[x]=0;
11        }
12    }
13    void push(ll x){cnt[x]=cnt[t[x][0]]+1+cnt[t[x][1]],vals[x]
    ↪ =vals[t[x][0]]+val[x]+vals[t[x][1]];}
14    void rotate(ll x){
15        ll y=f[x],z=f[y],d=dir(x);
16        t[y][d]=t[x][d^1],t[x][d^1]=y;
17        if(t[y][d]) f[t[y][d]]=y;
18        if(z) t[z][dir(y)]=x;
19        f[y]=x,f[x]=z,push(y),push(x);
20    }
21    void splay(ll &z,ll x){
22        ll w=f[z];
23        for(ll y;(y=f[x])!=w;rotate(x)){
24            if(f[y]!=w) rotate(dir(x)==dir(y)?y:x);
25        }
26        z=x;
27    }
28    void build(){
29        rt=0;
30        forn(i,1,n+3){
31            t[i][0]=rt;
32            if(rt) f[rt]=i;
33            rt=i;
34        }
35        splay(rt,1);
36    }
37    void loc(ll &z,ll k){
38        ll x=z;
39        for(push_down(x);cnt[t[x][0]]+1!=k;push_down(x)){
40            if(k>cnt[t[x][0]])
41                ↪ k-=cnt[t[x][0]]+1,x=t[x][1];
42            else x=t[x][0];
43        }
44        splay(z,x);
45    }
46    void opt(ll l,ll r){
47        loc(rt,l);
48        loc(t[rt][1],r-1+2);
49        ll x=t[t[rt][1]][0];
50        lz[x]^=1;
51    }
52    ll query(ll l,ll r){
53        loc(rt,l),loc(t[rt][1],r-1+2);
54        ll x=t[t[rt][1]][0],y=vals[x];
55        return y;
56    }
57    void solve(istream &cin){
58        cin >> n >> m;
59        forn(i,2,n+2) cin >> val[i];
60        build();
61        forn(i,0,m){
62            ll op,l,r; cin >> op >> l >> r;
63            if(op==1) opt(l,r);
64            else cout << query(l,r) << '\n';
65        }

```

6 Not that important

EPS.cpp

```

1  #include <cmath>
2  const double EPS = 1e-9;
3
4  bool eq(double a, double b) {
5      return fabs(a - b) < EPS;
6  }
7
8  bool lt(double a, double b) {

```

```

9      return a < b - EPS;
10 }
11
12 bool le(double a, double b) {
13     return a < b + EPS;
14 }

```

Gauce Elimination.cpp

```

1  // a[N][N] 是增广矩阵
2  int gauss()
3  {
4      int c, r;
5      for (c = 0, r = 0; c < n; c++)
6      {
7          int t = r;
8          for (int i = r; i < n; i++) //
9              ↪ 找到绝对值最大的行
10                 if (fabs(a[i][c]) > fabs(a[t][c]))
11                     t = i;
12
13             if (fabs(a[t][c]) < eps) continue;
14
15             for (int i = c; i <= n; i++) swap(a[t][i],
16                 ↪ a[r][i]); // 将绝对值最大的行换到最顶端
17             for (int i = n; i >= c; i--) a[r][i] /= a[r][c];
18             ↪ // 将当前行的首位变成1
19             for (int i = r + 1; i < n; i++) //
20                 ↪ 用当前行将下面所有的列消成0
21                 if (fabs(a[i][c]) > eps)
22                     for (int j = n; j >= c; j--)
23                         a[i][j] -= a[r][j] * a[i][c];
24
25             r++;
26         }
27
28         if (r < n)
29         {
30             for (int i = r; i < n; i++)
31                 if (fabs(a[i][n]) > eps)
32                     return 2; // 无解
33             return 1; // 有无穷多组解
34         }
35
36         for (int i = n - 1; i >= 0; i--)
37             for (int j = i + 1; j < n; j++)
38                 a[i][n] -= a[i][j] * a[j][n];
39
40         return 0; // 有唯一解
41     }

```

Joseph.cpp

```

1  int josephus(int n, int k) {
2      int res = 0;
3      for (int i = 1; i <= n; ++i) res = (res + k) % i;
4      return res;
5  }
6  //klogn version
7  int josephus(int n, int k) {
8      if (n == 1) return 0;
9      if (k == 1) return n - 1;
10     if (k > n) return (josephus(n - 1, k) + k) % n; //
11     ↪ 线性算法
12     int res = josephus(n - n / k, k);
13     res -= n % k;
14     if (res < 0)
15         res += n; // mod n
16     else
17         res += res / (k - 1); // 还原位置
18     return res;
19 }

```


7 New

Convex_hull_trick.cpp

```

1 class line{
2     public:
3         ll m,c;
4         line(){ }
5         line(ll m,ll c):m(m),c(c){ }
6         friend bool operator<(const line &a,const
        ↪ line &b){
7             return a.m<b.m || (a.m==b.m &&
        ↪ a.c>b.c);
8         }
9         ll cal(ll x){return m*x+c;}
10    };
11    constexpr ll mxN=1e5+1;
12    ll n,m,id=0,st[mxN];
13    vc<line> ln;
14    bool check(ll i,ll j,ll k){
15        return (ln[i].c-ln[k].c)*(ln[j].m-ln[i].m)<(ln[i]
        ↪ c-ln[j].c)*(ln[k].m-ln[i].m);
16    }
17    void solve(istream &cin){
18        cin >> n >> m;
19        forn(i,0,n){
20            ll y1,y2; cin >> y1 >> y2;
21            ln.emp(line{(y2-y1)/m,y1});
22        }
23        sort(all(ln),ln.resize(unique(all(ln),[](auto
        ↪ a,auto b){return a.m==b.m;})-ln.bg()));
24        forn(i,0,ln.size()){
25            while(id>1 && check(st[id-1],st[id],i))
        ↪ id--;
26            st[++id]=i;
27        }
28        ll ptr=1;
29        forn(i,0,m+1){
30            while(ptr+1<=id && ln[st[ptr+1]].cal(i)>ln
        ↪ [st[ptr]].cal(i))
        ↪ ptr++;
31            cout << ln[st[ptr]].cal(i) << " \n"[i==m];
32        }
33    }

```

Ray_casting.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){ }
5         Point(ll x,ll y):x(x),y(y){ }
6         friend Point operator-(const Point
        ↪ &a,const Point &b){return
        ↪ {a.x-b.x,a.y-b.y};}
7         friend ll operator*(const Point &a,const
        ↪ Point &b){return a.x*b.x+a.y*b.y;}
8         friend ll operator^(const Point &a,const
        ↪ Point &b){return a.x*b.y-a.y*b.x;}
9    };
10    ll n,m;
11    vc<Point> p;
12    bool on(Point &a,Point &b,Point &c){
13        if((b-a)^(c-a)) return false;
14        return 0<=((b-a)*(c-a)) &&
        ↪ ((b-a)*(c-a))<=((b-a)*(b-a));
15    }
16    void query(Point &x){
17        ll ok=0;
18        forn(i,0,n){
19            if(on(p[i],p[(i+1)%n],x)){ cout <<
        ↪ "BOUNDARY" << '\n';return;}
20            Point &a=p[i],&b=p[(i+1)%n];
21            if(a.y>x.y != b.y>x.y){
22                ll sg=(x-a)^(b-a);

```

```

        if(b.y>a.y==(sg<0)) ok^=1;
        }
    }
    cout << (ok&1?"INSIDE":"OUTSIDE") << '\n';
}

```

block_cut_tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 constexpr ll mxB=20;
3 ll n,m,q,st[mxN],id=0,mark[mxN]{},dfn[mxN]{},low[mxN],tim=
        ↪ 0,nid[mxN],pa[mxN<<1][mxB],d[mxN<<1];
4 vc<ll> adj[mxN];
5 vc<vc<ll>> cmp,t;
6 void tarjan(ll u=1,ll p=0){
7     dfn[u]=low[u]=++tim,st[++id]=u;
8     for(auto&v:adj[u]){
9         if(v==p) continue;
10        if(dfn[v]) low[u]=min(low[u],dfn[v]);
11        else{
12            tarjan(v,u);
13            low[u]=min(low[u],low[v]);
14            if(low[v]>=dfn[u]){
15                mark[u]=(dfn[u]>1 ||
        ↪ dfn[v]>2);
16                cmp.emp(vc<ll>{u});
17                while(cmp.back().back()!=v
        ↪ )
        ↪ cmp.back().emp(st[id--]);
        }
18        }
19    }
20    }
21    }
22    void dfs(ll u=1,ll p=0,ll lv=1){
23        d[u]=lv,pa[u][0]=p;
24        for(auto&v:t[u]){
25            if(v!=p) dfs(v,u,lv+1);
26        }
27    }
28    void built(){
29        ll x=0;
30        t.emp(vc<ll>{});
31        forn(i,1,n+1) if(mark[i])
        ↪ nid[i]=++x,t.emp(vc<ll>{});
32        for(auto&a:cmp){
33            ll y=++x;
34            t.emp(vc<ll>{});
35            for(auto&u:a){
36                if(mark[u]) t[nid[u]].emp(y),t[y].
        ↪ emp(nid[u]);
37                else nid[u]=y;
38            }
39        }
40        err(t.size());
41        dfs();
42        forn(j,1,mxB) forn(i,1,t.size())
        ↪ pa[i][j]=pa[pa[i][j-1]][j-1];
43    }
44    ll lca(ll a,ll b){
45        if(d[a]<d[b]) swap(a,b);
46        forr(i,mxB-1,0) if(d[a]-(1<<i)>=d[b]) a=pa[a][i];
47        if(a==b) return a;
48        forr(i,mxB-1,0) if(pa[a][i]!=pa[b][i])
        ↪ a=pa[a][i],b=pa[b][i];
49        return pa[a][0];
50    }
51    ll query(ll a,ll b,ll c){
52        ll lca1=lca(a,b),lca2=lca(a,c),lca3=lca(b,c);
53        return (lca1==c || (lca2==c && lca3==lca1) ||
        ↪ (lca3==c && lca2==lca1));
54    }
55    void solve(){
56        cin >> n >> m >> q;
57        forn(i,0,m){
58            ll u,v; cin >> u >> v;

```

```

59         adj[u].emp(v),adj[v].emp(u);
60     }
61     tarjan();
62     built();
63     while(q--){
64         ll a,b,c; cin >> a >> b >> c;
65         cout << (mark[c] &&
        ↪ query(nid[a],nid[b],nid[c])) || a==c
        ↪ || b==c?"NO":"YES" << '\n';
66     }
67 }

```

cross_dot_product.cpp

```

1 //A=(x1,y1),B=(x2,y2)
2 //A cross B = ABsin x = x1*y2-y1*x2
3 //A dot B = AB cos x = x1*x2+y1*y2

```

dec_string_to_bitset.cpp

```

1 // 10進位string(只含0-9, 長度<=500) 轉 bitset 二進位,
2 ↪ 支援先減一個小常數
3 // decSub(s,x): 回傳 s-x (x>=0, 需保證 s>=x), O(len)
4 // decToBits(s): 10進string -> bitset, b[0]為LSB, O(len*2)
5 // bitsToDec(b): bitset -> 10進string, 驗證/輸出用
6 // 500位 10進 1661 bits, mxB=2048 夠用
7 constexpr ll mxB=2048;
8 string decSub(string s,ll x){
9     for(ll i=s.size()-1;i>=0&&x;i--){
10         ll d=(s[i]-'0')-x%10; x/=10;
11         if(d<0){d+=10; x++;}
12         s[i]=d+'0';
13     }
14     ll p=0; while(p+1<(ll)s.size()&&s[p]=='0') p++;
15     return s.substr(p);
16 }
17 bitset<mxB> decToBits(string s){
18     bitset<mxB> b;
19     for(ll i=0;s!="0";i++){
20         b[i]=(s.back()-'0')&1;
21         ll r=0;
22         for(char &c:s){ll d=r*10+(c-'0');
23             ↪ c='0'+d/2; r=d&1;}
24         ll p=0; while(p+1<(ll)s.size()&&s[p]=='0')
25             ↪ p++;
26         s=s.substr(p);
27     }
28     return b;
29 }
30 string bitsToDec(bitset<mxB> b){
31     string s="0";
32     ll hi=mxB-1; while(hi>0&&!b[hi]) hi--;
33     forr(i,hi,0){
34         ll c=b[i];
35         forr(j,(ll)s.size()-1,0){ll
36             ↪ d=(s[j]-'0')*2+c; s[j]='0'+d%10;
37             ↪ c=d/10;}
38         if(c) s=char('0'+c)+s;
39     }
40     return s;
41 }
42 void solve(istream &cin){
43     string s; ll k;
44     cin>>s>>k; // 10進string 與 要減的常數
45     bitset<mxB> b=decToBits(decSub(s,k));
46     ll hi=mxB-1; while(hi>0&&!b[hi]) hi--;
47     forr(i,hi,0) cout<<b[i]; // MSB->LSB 印出 2進位
48     cout<<'\n';
49 }

```

dominator_tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,tim=0,id[mxN],sd[mxN],pa[mxN],f[mxN],lb[mxN],et[mxN]
3 ↪ []{},ret[mxN];
4 vc<ll> adj[mxN],radj[mxN],t[mxN],st;
5 void dfs(ll u=1){
6     et[u]=++tim,ret[tim]=u;
7     id[tim]=sd[tim]=lb[tim]=pa[tim]=tim;
8     for(auto&v:adj[u]){
9         if(!et[v]) dfs(v),f[et[v]]=et[u];
10         radj[et[v]].emp(et[u]);
11     }
12 }
13 ll find(ll u,ll x=0){
14     if(u==pa[u]) return x?-1:u;
15     ll v=find(pa[u],x+1);
16     if(v<0) return u;
17     if(sd[lb[u]]>sd[lb[pa[u]]]) lb[u]=lb[pa[u]];
18     pa[u]=v;
19     return x?v:lb[u];
20 }
21 void built(){
22     vc<ll> arr[n+1];
23     forr(i,n,1){
24         for(auto&v:radj[i])
25             ↪ sd[i]=min(sd[i],sd[find(v)]);
26         if(i>1) arr[sd[i]].emp(i);
27         for(auto&v:arr[i]){
28             ll x=find(v);
29             if(sd[x]==sd[v]) id[v]=sd[v];
30             else id[v]=x;
31         }
32         if(i>1) pa[i]=f[i];
33     }
34     forn(i,2,n+1){
35         if(id[i]!=sd[i]) id[i]=id[id[i]];
36         t[ret[i]].emp(ret[id[i]]),t[ret[id[i]]].em
37             ↪ p(ret[i]);
38     }
39 }
40 void dfs_ans(ll u=1,ll p=0){
41     st.emp(u);
42     if(u==n){
43         sort(all(st));
44         cout << st.size() << '\n';
45         for(auto&v:st) cout << v << ' '; cout <<
46             ↪ '\n';
47         exit(0);
48     }
49     for(auto&v:t[u]){
50         if(v==p) continue;
51         dfs_ans(v,u);
52     }
53     st.pop_back();
54 }
55 void solve(istream &cin){
56     cin>>n>>m;
57     forn(i,1,m+1){
58         ll u,v; cin>>u>>v;
59         adj[u].emp(v);
60     }
61     dfs();
62     built();
63     dfs_ans();
64 }

```

fft.cpp

```

1 class FFT{
2     public:
3         ll mod,root,root_inv,maX,M,M_inv;
4         FFT(){
5             FFT(ll a,ll b,ll c,ll d,ll e,ll
6                 ↪ f):mod(a),root(b),root_inv(c),maX(d),M
7                 ↪ (e),M_inv(f){}
8         }
9     };

```



```

6         ll fsp(ll a,ll m){
7             ll ans=1;
8             while(m){
9                 if(m&1) ans=(ans*a)%mod;
10                a=(a*a)%mod,m>=>=1;
11            }
12            return ans;
13        }
14        void fft(ll sz,vc<ll> &arr,ll inv){
15            for(ll i=1,j=0;i<sz;++i){
16                ll bit=sz>>1;
17                for(;j&bit;bit>=>=1)
18                    j^=bit; j^=bit;
19                if(i<j)
20                    swap(arr[i],arr[j]);
21            }
22            for(ll l=2;l<=sz;l<=>=1){
23                ll wlen=inv?root_inv:root;
24                for(ll i=1;i<=maX;i<=>=1)
25                    wlen=(wlen*wlen)%mod;
26                for(ll i=0;i<sz;i+=l){
27                    ll w=1;
28                    for(ll
29                        j=0;j<l/2;++j){
30                        ll x=arr[i+j]
31                        ll y=arr[i+j+l/2]
32                        ll z=(w*x-y)%mod;
33                        ll t=(x+y)%mod;
34                        arr[i+j]=t;
35                        arr[i+j+l/2]=z;
36                        w=w*wlen;
37                    }
38                }
39            }
40            if(inv){
41                ll n_1=fsp(sz,mod-2);
42                for(i=0,sz){
43                    arr[i]=(arr[i]*n_1)%mod;
44                }
45            }
46        }
47        constexpr ll MM=1002772198720536577;
48        //
49        //g=3
50        //998244353=119*2^23 + 1
51        //g^c=15311432
52        //inv=469870224
53        //M1=1004535809
54        //M1^-1=332747959
55        //
56        //1004535809=479*2^21 + 1
57        //g^c=702606812
58        //inv=700146880
59        //M2=998244353
60        //M2^-1=669690699
61        ll mul(ll a,ll b){
62            __int128 aa=a,bb=b,cc;
63            cc=(aa*bb)%MM;
64            return (ll)cc;
65        }
66        void solve(){
67            FFT fft1(998244353,15311432,469870224,1<<23,1004535809,669690699);
68            FFT fft2(1004535809,702606812,700146880,1<<21,998244353,669690699);

```

```

//ll x=(fft1.M*fft1.M_inv)%fft1.mod;
//ll y=(fft2.M*fft2.M_inv)%fft2.mod;
//err(x,y);
ll n,m;
cin >> n >> m;
vc<ll> a(n+1),b(m+1);
for(auto &v:a) cin >> v;
for(auto &v:b) cin >> v;
ll sz=1;
for(;sz<a.size()+b.size();sz<=>=1);
a.resize(sz),b.resize(sz);
vc<ll> A=a,B=b;
fft1.fft(sz,a,0);
fft1.fft(sz,b,0);
for(i=0,sz) a[i]=(a[i]*b[i])%fft1.mod;
fft1.fft(sz,a,1);

fft2.fft(sz,A,0);
fft2.fft(sz,B,0);
for(i=0,sz) A[i]=(A[i]*B[i])%fft2.mod;
fft2.fft(sz,A,1);

for(i=0,n+m+1) cout <<
    (mul(a[i],fft1.M*fft1.M_inv) +
    mul(A[i],fft2.M*fft2.M_inv))%MM << "
    \n"[i==n+m];
}

```

fwt.cpp

```

constexpr ll mxB=21;
ll a[1<<mxB]{};
void fwt(ll *arr,ll type){
    for(ll k=2;k<=(1<<mxB);k<=>=1){
        for(ll i=0,m=k>>1;i+m<(1<<mxB);i+=k){
            for(j=0,m){
                ll x,y; x=arr[i+j],y=arr[i+j+m];
                arr[i+j]=x+y,arr[i+j+m]=x-y;
                if(type) arr[i+j]/=2,arr[i+j+m]/=2;
            }
        }
    }
}
void solve(istream &cin){
    ll n,pfx,mx;
    cin>>n;
    mx=pfx=0;
    a[0]=1;
    for(i=0,n){
        ll x; cin>>x;
        pfx^=x,a[pfx]++,mx=max(mx,a[pfx]);
    }
    fwt(a,0);
    for(i=0,1<<mxB) a[i]=a[i]*a[i];
    fwt(a,1);
    ll res; res=0;
    // for(i=0,17) cout << a[i] << " \n"[i==16];
    for(i=0,1<<mxB) res+=(!i?mx>1:a[i]>0);
    cout << res << '\n';
    for(i=0,1<<mxB) if(!i?mx>1:a[i]) cout << i << '
    '; cout << '\n';
}

```

li_chao_tree.cpp

```

class line {
public:
    ll m,c;
    line(){m=0,c=-inf-1;}
    line(ll m,ll c):m(m),c(c){}
}

```

```

6         ll cal(ll x){return m*x+c;}
7     };
8     constexpr ll mxN=1e5+5;
9     ll n;
10    line t[mxN<<2];
11    void insert(line x,ll l,ll r,ll i,ll sl,ll sr){
12        if(sr<l || r<sl || l>r) return;
13        ll mid=(sl+sr)>>1;
14        if(l<=sl && sr<=r){ //if extended otherwise remove
15            ↪ this condition
16            bool lf=t[i].cal(sl-1)<x.cal(sl-1),mf=t[i].cal(
17            ↪ .cal(mid-1)<x.cal(mid-1),rf=t[i].cal(sr
18            ↪ r-1)<x.cal(sr-1);
19            if(mf) swap(x,t[i]);
20            if(sl==sr) return;
21            if(lf^mf) insert(x,l,r,i<<1,sl,mid);
22            if(rf^mf) insert(x,l,r,i<<1|1,mid+1,sr);
23            return;
24        }
25        insert(x,l,r,i<<1,sl,mid);
26        insert(x,l,r,i<<1|1,mid+1,sr);
27    }
28    ll query(ll x,ll i,ll sl,ll sr){
29        ll mid=(sl+sr)>>1;
30        if(sl==sr) return t[i].cal(x-1);
31        if(x<=mid) return
32            ↪ max(t[i].cal(x-1),query(x,i<<1,sl,mid));
33        else return
34            ↪ max(t[i].cal(x-1),query(x,i<<1|1,mid+1,sr));
35    }
36    void solve(){
37        cin >> n;
38        forn(i,0,n){
39            ll op;
40            cin >> op;
41            if(op==1){
42                ll a,b,l,r;
43                cin >> a >> b >> l >> r;
44                line x{a,b};
45                insert(x,l+1,r+1,1,1,mxN-4);
46            }
47            else{
48                ll x;
49                cin >> x;
50                x=query(x+1,1,1,mxN-4);
51                if(x>-inf-1) cout << x << '\n';
52                else cout << "NO" << '\n';
53            }
54        }
55    }

```

shoelace.cpp

```

1 //consecutive point p[i] (0<=i<n)
2 //area= p[i].x*p[(i+1)%n].y - p[i].y*p[(i+1)%n].x
3
4 void solve(istream &cin){
5     ll n,res=0;
6     cin>>n;
7     forn(i,1,n+1){
8         cin>>a[i];
9         if(i>1) res+=a[i-1]^a[i]; //cross product
10        if(i==n) res+=a[n]^a[1];
11    }
12    cout << abs(res) << '\n';
13 }

```

suffix_automaton.cpp

```

1 constexpr ll mxN=1e5+1;
2 class state{
3     public:
4         ll len,link,next[26];
5

```

```

6     state(){
7     };
8     ll n,sz;
9     state t[mxN<<1];
10    void sam(const string &s){
11        memset(t,0,sizeof(t));
12        t[0].link=-1;
13        ll last,cur; last=0;
14        for(auto&v:s){
15            cur++;sz;
16            t[cur].len=t[last].len+1;
17            ll p; p=last;
18            while(p!=-1 && !t[p].next[v-'a'])
19                ↪ t[p].next[v-'a']=cur,p=t[p].link;
20            if(p==-1) t[cur].link=0;
21            else{
22                ll q; q=t[p].next[v-'a'];
23                if(t[p].len+1==t[q].len) t[cur].link=q;
24                else{
25                    ll clone; clone++;sz;
26                    t[clone].len=t[p].len+1,t[clone].link=t[q].
27                    ↪ .link;
28                    memcpy(t[clone].next,t[q].next,sizeof(t[q].
29                    ↪ .next));
30                    while(p!=-1 && t[p].next[v-'a']==q)
31                        ↪ t[p].next[v-'a']=clone,p=t[p].link;
32                    t[q].link=t[cur].link=clone;
33                }
34            }
35            last=cur;
36        }
37    }
38    void solve(istream &cin){
39        string s;
40        cin>>s;
41        n=s.length(),sz=0;
42        sam(s);
43        vc<ll> res(n+2,0);
44        forn(i,1,sz+1)
45            ↪ res[t[t[i].link].len+1]++,res[t[i].len+1]--;
46        partial_sum(all(res),res.bg());
47        forn(i,1,n+1) cout << res[i] << " \n"[i==n];
48    }

```

suffixarray_and_lcp.cpp

```

1 constexpr ll mxN=1e5+5;
2 ll sa[mxN],nsa[mxN],c[mxN],nc[mxN],cnt[mxN] {},lcp[mxN];
3 void sfa(string s){
4     s+="$";
5     ll n=s.length();
6     forn(i,0,n) cnt[s[i]]++; forn(i,1,256)
7         ↪ cnt[i]+=cnt[i-1];
8     forr(i,n-1,0) sa[--cnt[s[i]]]=i; c[sa[0]]=0;
9     ↪ forn(i,1,n)
10        ↪ c[sa[i]]=c[sa[i-1]]+(s[sa[i]]!=s[sa[i-1]]);
11    for(ll h=0;(1<=h)<n;h++){
12        forn(i,0,n) nsa[i]=((sa[i]-(1<=h))%n+n)%n;
13        fill(cnt,cnt+c[sa[n-1]]+1,0);
14        forn(i,0,n) cnt[c[nsa[i]]]++;
15        ↪ forn(i,1,c[sa[n-1]]+1)
16            ↪ cnt[i]+=cnt[i-1];
17        forr(i,n-1,0) sa[--cnt[c[nsa[i]]]]=nsa[i];
18        nc[sa[0]]=0;
19        forn(i,1,n){
20            pll pre={c[sa[i-1]],c[(sa[i-1]+(1<
21            ↪ <h))%n]},cur={c[sa[i]],c[(sa[i]
22            ↪ ]+(1<=h))%n]};
23            nc[sa[i]]=nc[sa[i-1]]+(cur!=pre);
24        }
25        swap(c,nc);
26    }
27 }
28 void Lcp(const string &s){
29     ll n=s.length(),k=0;
30     forn(i,0,n){

```

```

24         if(c[i]>n-1){k=0; continue;}
25         ll j=sa[c[i]+1];
26         for(;i+k<n && j+k<n && s[i+k]==s[j+k];k++){
27             lcp[c[i]]=k;
28             if(k) k--;
29         }
30     }

```

xor_basis.cpp

```

1  constexpr ll mxN=2e5+1;
2  constexpr ll mxB=31;
3  ll n,a[mxN],b[mxB]{};
4  void built(){
5      forn(i,1,n+1){
6          ll x; x=a[i];
7          forr(j,30,0){
8              if((x>>j)&1){
9                  if(!b[j]){b[j]=x; break;}
10                 else x^=b[j];
11             }
12         }
13     }
14 }
15 void solve(istream &cin){
16     cin>>n;
17     forn(i,1,n+1) cin>>a[i];
18     built();
19     ll res; res=0;
20     forr(i,30,0){

```

```

21         if(res>>i&1^1) res^=b[i];
22     }
23     cout << res << '\n';
24 }

```

zfunction_and_kmp.cpp

```

1  constexpr ll mxN=1e6+1;
2  vc<ll> zf(const string &s){
3      ll n,l,r; n=s.length(),l=r=-1;
4      vc<ll> z(n,0);
5      forn(i,1,n){
6          if(r>i) z[i]=min(z[i-1],r-i);
7          while(i+z[i]<n && s[i+z[i]]==s[z[i]])
8              z[i]++;
9          if(i+z[i]>r) r=i+z[i],l=i;
10     }
11     return z;
12 }
13 vc<ll> kmp(const string &s){
14     ll n; n=s.length();
15     vc<ll> k(n,0);
16     forn(i,1,n){
17         ll j; j=k[i-1];
18         while(j>0 && s[i]!=s[j]) j=k[j-1];
19         if(s[i]==s[j]) k[i]=j+1;
20     }
21     return k;

```